

LUNCH AND LEARN • FIVE PRACTICES • 60 MINUTES

Agentic engineering

Coding agents are fast. These five practices make them reliable.

THE ART OF THE POSSIBLE

Agents can now take a ticket all the way to review.

STEP 01

Plan

Reads the ticket and plans the change.

STEP 02

Build

Writes code and tests, opens a pull request.

STEP 03

Fix

Fixes whatever breaks the build.

STEP 04

Report

Notes how sure it is, and why.

STEP 05

Hand off

Waits for approval, leaves notes.

SO WHAT

The bottleneck has moved from writing the code to trusting the result.

THE ONE RULE • USE AGENTS ANY WAY YOU LIKE

Reality must match the claim

AI raises your floor, but only you can raise your ceiling.

THE FLOOR

Anyone can now produce working-looking code in minutes. That part got cheap, for everyone.



THE CEILING

Knowing what to build, spotting what is wrong, deciding when it is good enough. That part is still yours.

SO WHAT

Everyone has a camera and a crew now. Not everyone makes a film.

YOUR ROLE • THE DIRECTOR

Direct the work: watch the take, name the problem, give one note.

STEP 01

Watch the take

Read the actual change and run it before you react.

STEP 02

Name the problem

Say exactly what is off: "the retry hides the real error."

STEP 03

Give one note

Say it once, clearly, then let the agent work.

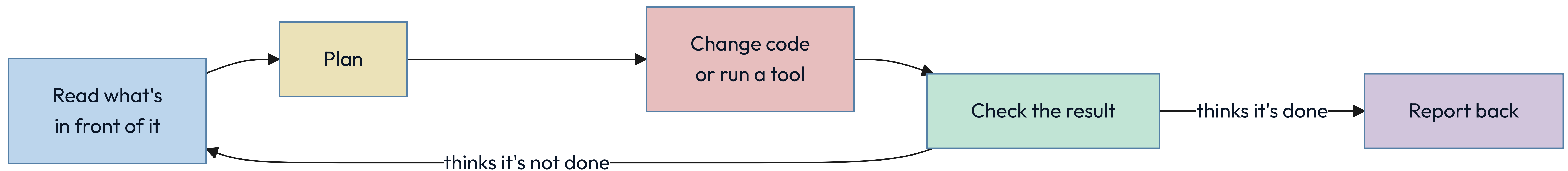
YOUR ROLE • THE SET

Everything on a film set has a counterpart in agentic work.

ON SET	WITH AGENTS	PRACTICE
Script and shot list	The plan and the instruction file	Context
One clear note per take	One precise request	Autonomy
The dailies	Proof from the real thing	Verification
Continuity	Checks that catch junk before it ships	Verification
Production notes	Decision log and follow-up files	Learning
Casting and the final cut	An agent roster; you approve the merge	Orchestration

HOW AN AGENT WORKS

A coding agent loops until it thinks it is done.



TAKEAWAY

Every practice today makes "thinks it is done" match "is done."

Five practices, one for each question you will face.

01 Context: control what the agent sees

02 Autonomy: decide what it may do alone

03 Verification: make every claim prove itself

04 Learning: turn mistakes into rules

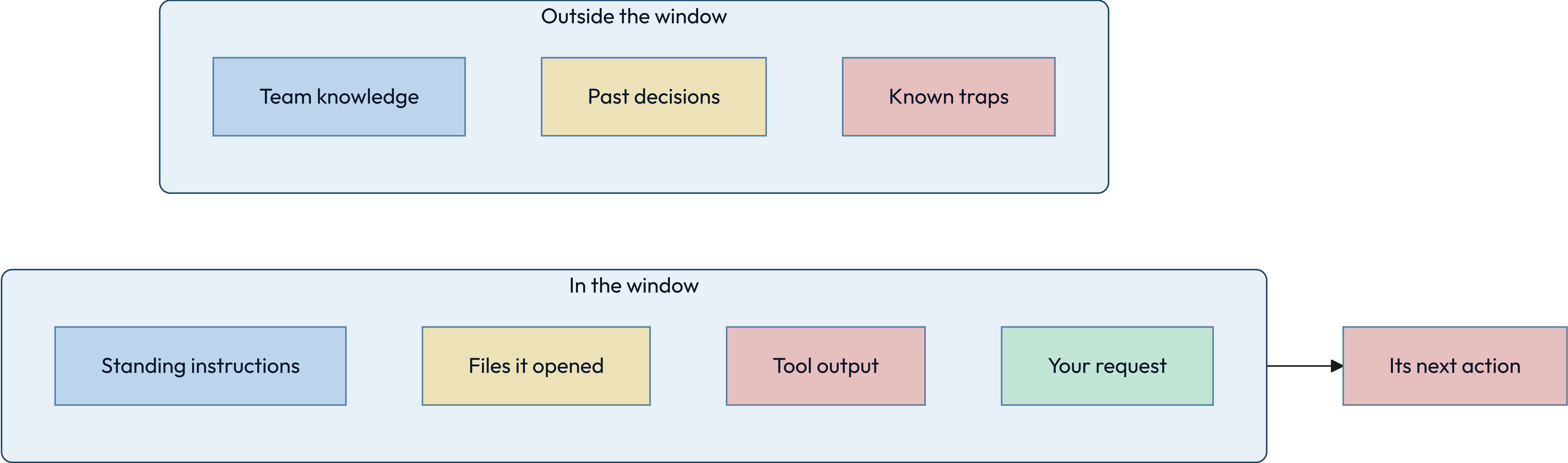
05 Orchestration: use many agents without losing control

PRACTICE 01 · THE SCRIPT AND THE SHOT LIST

Context engineering

CONTEXT • THE CORE IDEA

An agent sees only its context window, the text in front of it.



IMPLICATION

What you put in this window decides most of the agent's quality.

Write the always-loaded file as an index, and keep the manual elsewhere.

A MANUAL

Every rule with its full explanation. It grows with each incident, and every session pays to read all of it.



AN INDEX

One line per rule and a link to the document that explains it. The agent opens the detail only when the task needs it.

Route the agent to the right document before it starts work.

Read these before working in an area

Working on...	Read first
-----	-----
Branching, merging, releases	engineering/workflow.md
Tests, hooks, CI	engineering/development.md
Something behaving strangely	engineering/gotchas.md
Building a new script	engineering/capabilities.md
A past decision	engineering/decisions/

Four habits keep the window full of what matters.

Read sections, not files: list the headings, then open only what you need.

Delegate big reads: a helper agent reads the log and returns a summary.

Quiet the tools: print failures in full and successes as a dot.

Measure first: check what real sessions load before you optimize.

CONTEXT • A QUIETER TEST REPORT

99.8%

less for the agent to read per test run:

657,806 tokens down to 1,182.

Claude Code gives you a setting for each of these habits.

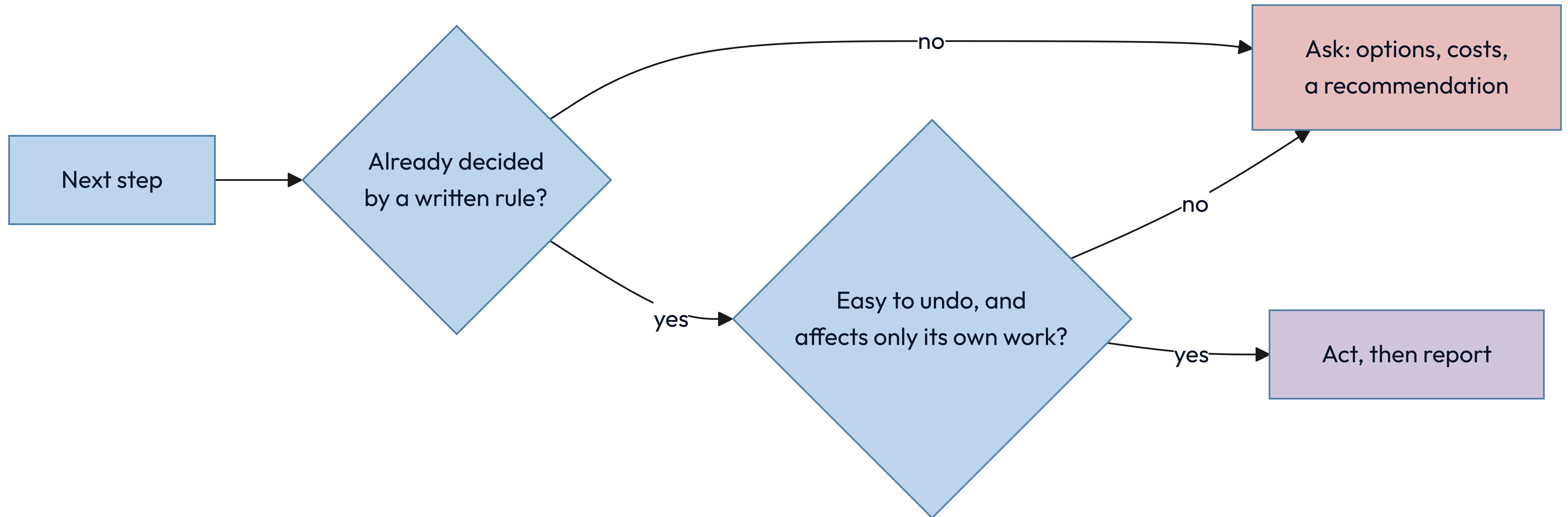
PRACTICE	IN CLAUDE CODE
Standing instructions as an index	<code>CLAUDE.md</code> , linking detail docs with <code>@docs/file.md</code>
Guidance that loads only when relevant	A skill in <code>.claude/skills/<name>/SKILL.md</code>
Big reads outside the main session	A helper agent (subagent) in <code>.claude/agents/</code>
See what fills the window	<code>/context</code> , and <code>/usage</code> for the bill

PRACTICE 02 • WHAT THE CREW DECIDES ALONE

Autonomy with limits

AUTONOMY · TWO QUESTIONS

Two questions decide whether the agent acts or asks.



Risk comes from what you can't undo and who else it affects.

Easy to undo · Only its own work.

Code, tests, docs

The agent acts

Easy to undo · Reaches others.

Shared labels, team settings

The agent proposes

Hard to undo · Only its own work.

Deleted work, exported files

The agent shows the result first

Hard to undo · Reaches others.

Merges, releases, publishing

A person decides

WHY IT MATTERS

Others act on shared state before you can undo a mistake there.

Five kinds of change always come back to a person.

Shared state: labels, boards and settings other people read.

The build pipeline: every future change pays for a new step.

A number a person set: we asked for twelve and got sixty.

A core document's meaning: rewriting rules differs from following them.

Anything irreversible or public: merges, releases, comments on others' work.

When the agent asks, it brings measured options and a recommendation.

A WEAK QUESTION

"Should I add this check to the build?" Now you have to do the analysis.



A STRONG QUESTION

"This check adds half a second to each build and catches the bug we hit last week. I recommend adding it." You just decide.

Permissions and plan mode turn the reach test into settings.

PRACTICE

IN CLAUDE CODE

Run safe commands freely, never risky ones

The allow, ask and deny lists in `.claude/settings.json`

Show the plan before touching anything

Plan mode: `Shift+Tab`

The same limits for every developer

Managed settings, rolled out org-wide

PRACTICE 03 • WATCH THE DAILIES

Verification you can trust

Agents report what they believe, and that can differ from what is true.

WHAT THE AGENT REPORTED

"Verified with a real browser," in the pull request, the commit message and the design note.



WHAT WAS TRUE

The test still passed with the fix deleted. It could never fail, so it proved nothing.

BOTTOM LINE

Rules an agent checks on itself are weak. A second, independent check catches what they miss.

VERIFICATION • THE CLAIM RULE

Every "verified" says where it ran and shows proof from there.

Say where: the real browser, the real export, the real device.

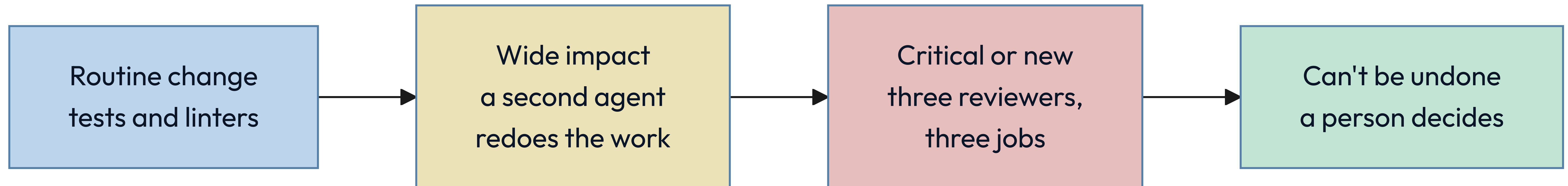
Show proof: a screenshot, a log or a number from that place.

Say "unverified" out loud when the real thing is out of reach.

Treat a passing build as partial: it proves only what the build runs.

VERIFICATION • THE LADDER

Match the amount of checking to the damage a mistake could do.



Three reviewers with different jobs catch different failures.

Red team

Tries to break it: edge cases, misuse, the input nobody tried.

Skeptic

Asks whether we are solving the right problem at all.

Fact checker

Re-derives every fact and number from the source.

OUR VIEW

Reviewers who check correctness miss a wrong goal. Give that job to one reviewer.

A check earns trust by proving it can fail.

Reasons for exceptions

Every exception to a rule has a written reason.

Unused exceptions fail

An exception that no longer applies breaks the build.

Plant a known bug

Each run plants a bug and fails if the check misses it.

Warn, then block

A new check blocks merges only after a clean record.

WHY IT MATTERS

A check that always passes looks exactly like a check that works.

Beyond unit tests, each kind of test answers a different question.

TEST	THE QUESTION IT ANSWERS
Mutation test	If I break the code on purpose, does a test fail?
Metamorphic test	If I change the input in a known way, does the output change the way it should?
Visual diff	Does the output look the same as before, unless I meant it to change?
Benchmark against a baseline	Did it get slower, and by how much?
Fuzz test	Does strange or random input break it?

VERIFICATION • TAUTOLOGICAL TESTS

A test that repeats the code's own math can't catch its mistakes.

REPEATS THE CODE

```
test('adds sales tax', () => {  
  // Same formula as the code under test.  
  // If the rate is wrong, both are wrong,  
  // and this still passes.  
  expect(tax(100)).toBe(100 * TAX_RATE);  
});
```

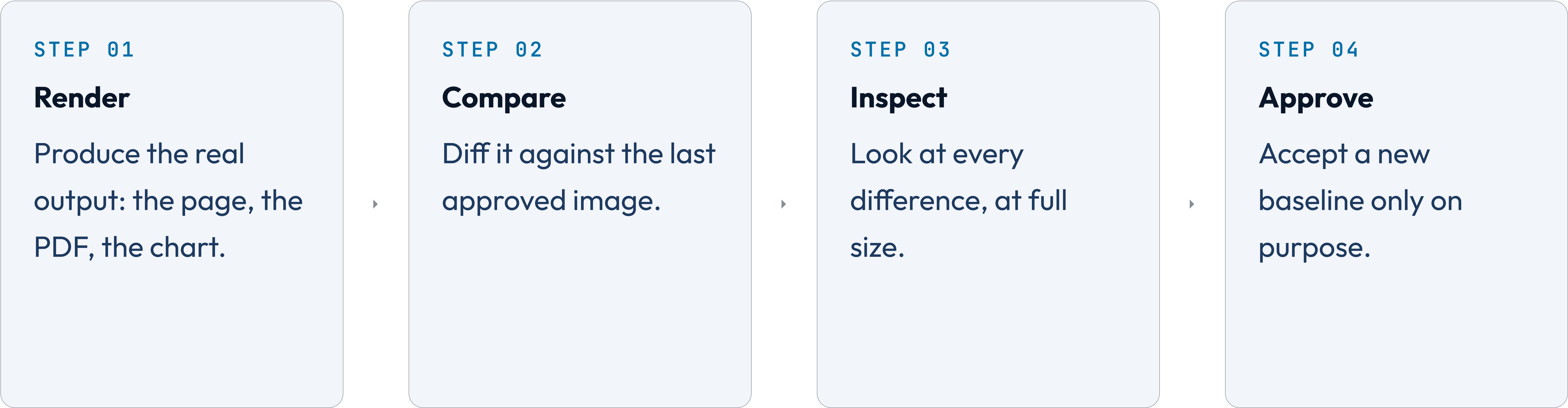
CHECKS A KNOWN ANSWER

```
test('adds sales tax', () => {  
  // Worked out by hand from the tax table.  
  // If the code is wrong, this fails.  
  expect(tax(100)).toBe(8.25);  
});
```

WHY IT MATTERS

Agents write tests that pass. Your job is to make sure they can fail.

Nothing proves it looks right like rendering it and looking.



OUR VIEW

Tests check what you thought to check. A rendered picture shows everything.

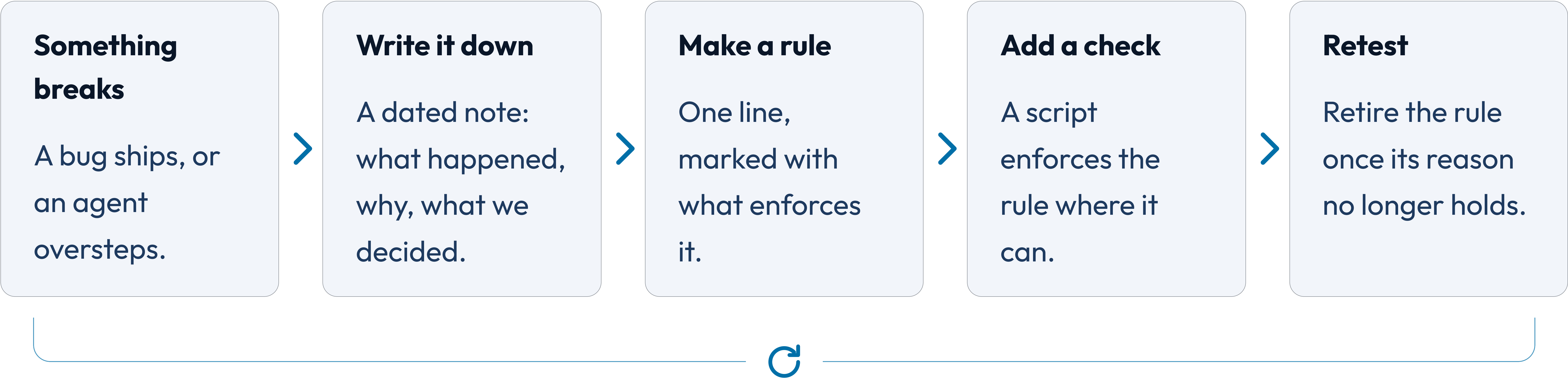
Hooks and helper agents run the checks without being asked.

PRACTICE	IN CLAUDE CODE
Run checks after every edit	A hook that runs after each edit (<code>PostToolUse</code>)
Refuse "done" until the tests pass	A hook that runs when it tries to finish (<code>Stop</code> , exit code 2)
An independent reviewer	A read-only helper agent, or <code>/code-review</code>

PRACTICE 04 • KEEP CONTINUITY

A system that learns

Every rule should travel one loop, from mistake to retirement.



KEY INSIGHT

Copy the loop, and let your own mistakes write your rules.

A decision note records why, so nobody has to guess later.

```
---  
status: shipped  
summary: Allow real database calls in tests; the ban had no evidence  
---
```

```
# Retire the rule against database calls in tests
```

Symptom	The rule forced slow, fragile mocks for two months
Claim	Real database calls make tests flaky
Evidence	500 runs against a test database: 0 flaky
Decision	Retire the rule; keep its number unused

Proposals, decision records and specs answer different questions.

DOCUMENT	ANSWERS	WHEN IT EXPIRES
Proposal	Which option should we pick?	Once someone decides
Decision record	Why is it this way?	Never; a newer record replaces it
Spec	What must every implementation do?	Never; you edit it to stay true

Before every merge, the agent grades its confidence by its weakest point.

Pre-merge: add retry to failed exports

WHAT One automatic retry on a failed export

EVIDENCE Real export run 20 times: 0 failures

RISK Export path only · revert: one commit

UNVERIFIED Safari's download dialog

CONFIDENCE high, limited by the evidence

raise it by: run the export in Safari

RECOMMENDATION

Grade by the weakest area, and always name the one thing that would raise it.

Keep pending work in the repository, one file per item.

IN CHAT

Gone when the session ends. When we finally checked, nearly half of our chat-only to-dos were already done or duplicated.



IN THE REPOSITORY

One small file per item, with a priority and a "done when." Any session can pick it up cold.

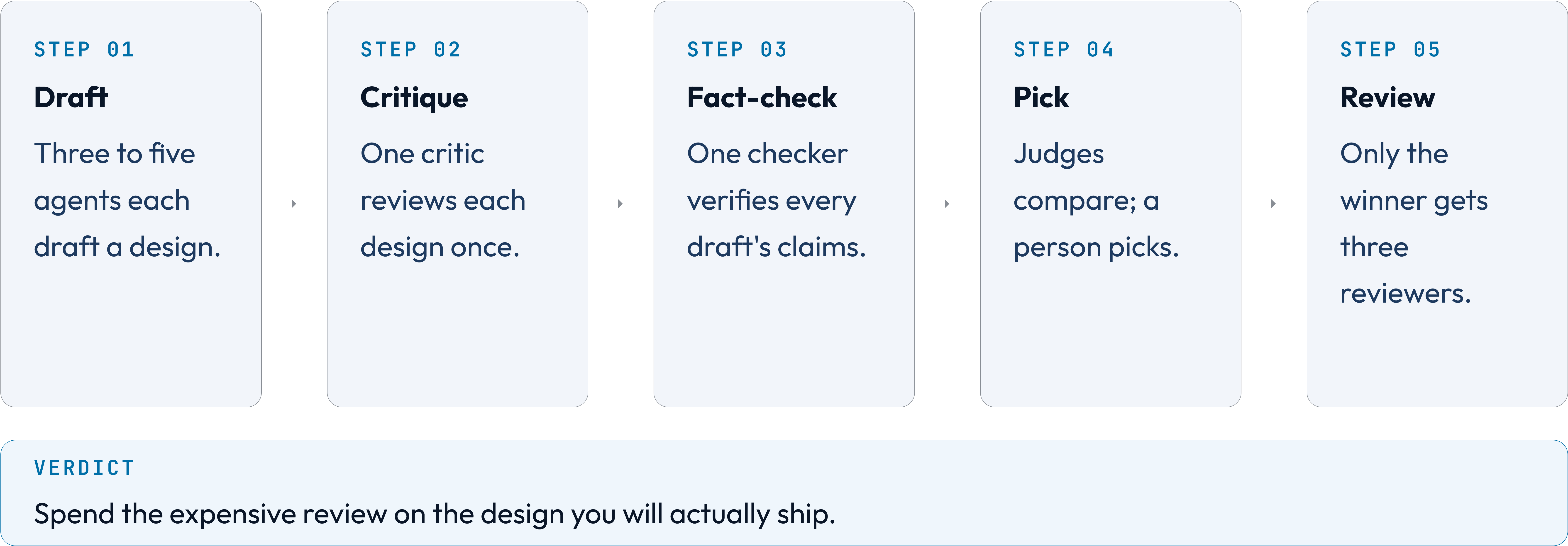
PRACTICE 05 • CAST THE CREW

Orchestration

Give each agent one job and a name, and pick it by its job.

AGENT	ITS ONE JOB
Scout	Find where things live and how they work
Fact checker	Confirm or refute each claim against the source
Build fixer	Find why a check failed, and fix the cause
Red team	Break the change before users do
Skeptic	Ask whether we are solving the right problem
Prose checker	Catch unclear or machine-sounding writing

Compare several drafts, then review only the winner.



Treat agent count like money: estimate it, cap it, and stop early.

01 Estimate how many agents and roughly what it will cost before you start.

02 Count across the whole session; small runs add up.

03 Past about ten agents, a person approves.

04 Stop when a round changes nothing, usually by round three.

05 Record what it cost, so you learn what it was worth.

Start with one habit per practice, then add a layer each month.

PRACTICE	WEEK 1	MONTH 1	QUARTER 1
Context	☐ A one-page index file	☐ Linked detail docs	☐ Measure what sessions load
Autonomy	☐ Allow and deny lists	☐ A written stop list	☐ Human approval in the platform
Verification	☐ Tests on every change	☐ A check that plants a bug	☐ A second agent on risky work
Learning	☐ A decision log	☐ Pending work as files	☐ Retest your oldest rules

☐ Next step

The practices carry over, but what counts as "the real thing" changes.

WORK	THE REAL THING TO CHECK	A CHECK THAT PROVES IT CAN FAIL
Data science	A fixed, versioned evaluation set	Plant an input that leaks the answer
Data engineering	The target warehouse	Insert a known-bad row
Analytics and BI	The published dashboard	Plant a mismatch with the source
Services	Staging, or a slice of live traffic	Break a contract on purpose
Tools and CLIs	The built program's output	A saved expected output that must change

THE ONE RULE • WHY IT COMPOUNDS

Agents copy what they find, so quality and jank both compound.

FIX IT IN POST

The next agent reads the shortcut as the house style and copies it. A month later, it is in ten files.



GET IT RIGHT ON SET

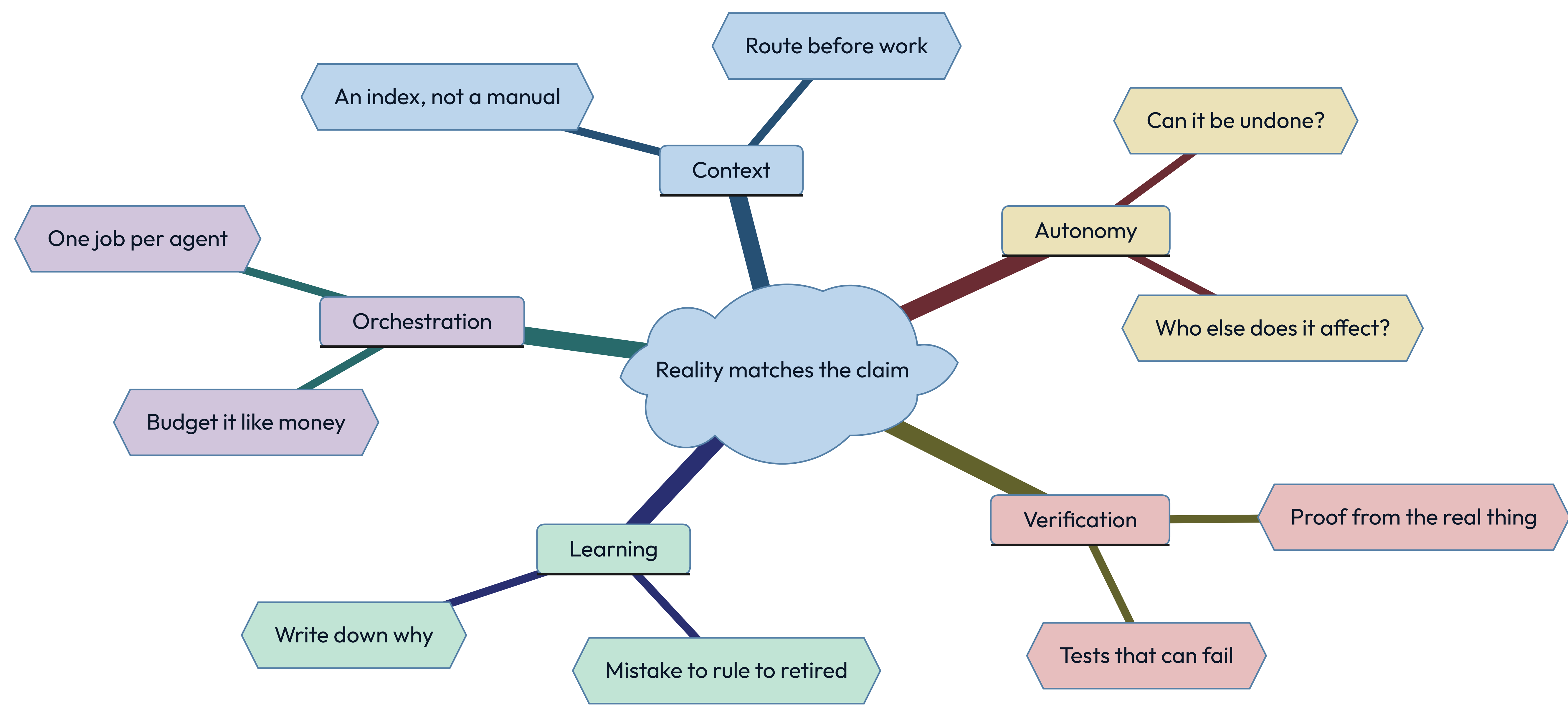
The next agent copies the clean version instead. Every change starts from firmer ground.

WHY IT MATTERS

Every shortcut you ship becomes context for the next agent.

THE BIG PICTURE · ONE RULE, FIVE PRACTICES

You stop typing code and start directing a system.



YOUR NEXT STEP

Three moves this week put all five practices in motion.

01 Rewrite your instruction file as an index, one line per rule.

02 Add one check, then break your code on purpose and watch it catch the bug.

03 Start a decision log: one dated note each time something goes wrong.

THE ASK

Pick one move and try it before next Friday.

QUESTIONS

Make reality match the claim

Quality compounds. So does jank.

THE STARTER KIT

Copy these, then make them yours

Seven terms from this talk, in plain words.

C – T

TERM	DEFINITION
CI, the build	Automated checks that run on every proposed change.
Context window	Everything the model can see at one moment.
Hook	A script the agent tool runs at a fixed moment.
Pull request	A proposed change, waiting for review before it merges.
Session	One conversation with an agent, from start to finish.
Tautological test	A test that can only agree with the code it checks.
Token	A chunk of text, about three quarters of a word.

Seven files put all five practices into a repository in five minutes.

FILE	WHERE IT GOES	PRACTICE
Instruction file	CLAUDE.md	Context, autonomy
Settings	.claude/settings.json	Autonomy, verification
Stop hook	.claude/hooks/	Verification
Reviewer agent	.claude/agents/checker.md	Verification, orchestration
Evidence card	PR template	Learning
Decision note, follow-up file	docs/decisions/, followups/	Learning

The instruction file states the rules once and links to the detail.

Always

- Run the tests before you say anything is done.
- Failing test first, then the fix, then the same test passing.
- "Verified" says where it ran and shows proof from there.
Otherwise, write UNVERIFIED.

Ask first, even when a rule points at it

- Shared state: labels, boards, settings others read.
- The CI pipeline or git hooks.
- A number a person set: "about 12" is a decision.
- The meaning of a core doc, including this file.
- Anything irreversible or public.

The settings file turns the reach test into allow, ask and deny lists.

```
{
  "permissions": {
    "allow": ["Bash(npm test*)", "Bash(git diff*)"],
    "ask":   ["Bash(git push*)"],
    "deny":  ["Bash(git push --force*)", "Read(./env)"]
  },
  "hooks": {
    "Stop": [{ "hooks": [{ "type": "command",
      "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/tests-must-pass.sh"
    }] }]
  }
}
```

The stop hook keeps the agent working until the tests pass.

```
#!/usr/bin/env bash
# Exit code 2 sends stderr back to Claude and keeps it working.
input=$(cat)
# Already sent back once by this hook? Let it stop.
echo "$input" | grep -q '"stop_hook_active": *true' && exit 0

log=$(mktemp)
if ! npm test --silent >"$log" 2>&1; then
    echo "Tests are failing. Fix them before you finish:" >&2
    tail -20 "$log" >&2
    exit 2
fi
```

The reviewer agent re-derives every claim and never edits.

```
---  
name: checker  
description: Independent reviewer with fresh eyes. Re-derives  
  every claim and "verified" in a change. Never edits.  
tools: Read, Grep, Glob, Bash  
---  
You are a skeptical reviewer who did not write this change.  
For each claim: find the evidence yourself, run what can be  
run, and check that a test fails when the fix is removed.  
Mark each CONFIRMED, REFUTED or UNVERIFIABLE, with proof.  
Treat "I believe it works" and "CI is green" as unverified.
```

The evidence card puts the facts in front of every merge decision.

```
Pre-merge: <PR title>
WHAT      <one line: what actually lands>
WHY       <one line: the problem it solves>
EVIDENCE  <what was run or measured, and where>
RISK      <what breaks if wrong> • revert: <how>
UNVERIFIED <caveats that bear on this decision, or none>
CONFIDENCE <low | medium | high | very high>: <weakest area>
          raise it by: <the one thing that would raise it>
```

Areas: evidence • reach • undo • unknowns • review

The decision note records why, and when to check it again.

```
---
status: proposed      # proposed | shipped | superseded
type: decision        # proposal | decision | spec | scoping
summary: One line a reader can scan in the index
---
# <The decision, as a sentence>

Symptom.      What prompted this, with a link or number.
Cause.        Why. Measured, observed or argued?
Decision.     What we will do, and what we rejected.
Enforced by.  The check that holds it, or "discipline only".
Retest when.  When we check that this still holds.
```

The follow-up file keeps pending work where the next session will find it.

```
---  
origin: 1234  
priority: P1  
recorded: 2026-09-25  
---  
# Export drops the last row when the file ends without a newline  
  
why now    – every weekly report is one row short  
where      – src/export/csv.ts  
done when  – a file with no trailing newline exports every row  
verify     – the new unit test, then one real export
```